Introduction to Assembly Language Programming (8086)

1. What is Assembly Language?

Assembly language is a **low-level programming language** that is closely related to machine language. While machine language consists of binary instructions (0s and 1s), assembly language uses **mnemonics (readable instructions)** to make programming easier for humans.

Example:

- Machine code: 10110000 01100001
- Assembly: MOV AL, 61H

Both perform the same operation, but assembly is easier to understand.

2. Role of CPU in Assembly Programming

The CPU (Central Processing Unit) performs three main tasks:

1. Takes input
2. Processes data
3. Produces output

In assembly language, we directly control the CPU using instructions.

3. Registers in 8086

Registers are small, fast storage locations inside the CPU. They are used to hold data temporarily during execution.

- 8086 is a **16-bit processor**, so most registers are 16-bit (can hold 2 bytes).

2. Classification of 8086 Registers

Registers are divided into **4 main groups**:

1. General Purpose Registers
2. Segment Registers
3. Pointer & Index Registers
4. Flag Register

3. General Purpose Registers

These are used for data manipulation and arithmetic operations.

Register	Size	Description
AX		16-bit Accumulator (main register for operations)
BX		16-bit Base register (used in memory addressing)
CX		16-bit Counter register (used in loops)
DX		16-bit Data register (used in I/O operations)

3.1 AX (Accumulator Register)

Most important register

- Used in arithmetic operations
- Used with I/O instructions
- Can be divided into:
    - AH (high 8-bit)
    - AL (low 8-bit)

Example:

MOV AL, 5

MOV AH, 2

3.2 BX (Base Register)

Used for memory addressing

MOV BX, 1000H

MOV AL, [BX]

3.3 CX (Counter Register)

Used in loops and counting

MOV CX, 5

LOOP START

3.4 DX (Data Register)

Used in multiplication, division, and I/O

OUT 80H, AL

Sometimes DX holds port address in advanced cases

4. Segment Registers

Used to store **memory segment addresses**.

Register	Description
CS	Code Segment
DS	Data Segment
SS	Stack Segment
ES	Extra Segment

4.1 CS (Code Segment)

Points to the code being executed

4.2 DS (Data Segment) : Points to data memory

4.3 SS (Stack Segment) : Points to stack memory

4.4 ES (Extra Segment)

Additional data segment

5. Pointer and Index Registers

Used for **memory access and addressing**

Register	Description
SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Destination Index

5.1 SP (Stack Pointer)

Points to top of stack

5.2 BP (Base Pointer): Used to access stack data

5.3 SI (Source Index)

Used in string operations (source)

5.4 DI (Destination Index)

Used in string operations (destination)

6. Instruction Pointer (IP)

Holds address of next instruction to execute

- Works with CS register
- Automatically updated by CPU

7. Flag Register

16-bit register that stores **status flags**

Important Flags

Flag	Meaning
CF	Carry Flag
ZF	Zero Flag
SF	Sign Flag
OF	Overflow Flag

Flag	Meaning
PF	Parity Flag
AF	Auxiliary Carry

Example

MOV AL, 5

SUB AL, 5

Result = 0 → ZF = 1

8. Register Summary Diagram

AX = AH + AL

BX = BH + BL

CX = CH + CL

DX = DH + DL

- AL is used in IN/OUT instructions
- AH is used for INT 21H function selection
- DL is used for character output
- CX is used for loop control
- AX → main operations
- BX → base addressing
- CX → loop counter
- DX → I/O operations
- CS/DS/SS → memory segments
- SP/BP/SI/DI → addressing
- Flags → condition checking

Important Registers:

Register	Description
AL	8-bit data register
AH	High byte of AX / function selector
AX	16-bit register (AH + AL)
DL	Used for output
CX	Loop counter

4. Basic Instructions

Assembly language consists of instructions that tell the CPU what to do.

4.1 MOV (Data Transfer)

Transfers data from source to destination.

```
MOV AL, 5
```

4.2 ADD (Addition)

```
ADD AL, 3
```

4.3 SUB (Subtraction)

```
SUB AL, 2
```

4.4 INC / DEC (Increment / Decrement)

```
INC AL   ; AL = AL + 1
DEC AL   ; AL = AL - 1
```

5. Interrupts (INT 21H)

Interrupts allow the program to communicate with the operating system (DOS).

```
INT 21H
```

The function performed depends on the value stored in the **AH register**.

Important Functions:

AH Value	Function
01H	Input a character from keyboard

AH Value	Function
02H	Output a character to screen
09H	Display string

Example:

MOV AH, 01H

INT 21H

This takes input from the keyboard and stores it in the **AL register**.

6. Input and Output Flow

Keyboard → AL → DL → Screen

- Input is stored in **AL**
- Output is taken from **DL**

7. Loop

Loops are used to repeat instructions.

MOV CX, 5

START:

; instructions

LOOP START

The loop runs until **CX becomes zero**.

8. Interfacing (I/O Ports)

In interfacing, the CPU communicates with external devices using **I/O ports**.

8.1 OUT Instruction (Send Data)

OUT 80H, AL

Sends data from AL register to port 80H.

8.2 IN Instruction (Receive Data)

IN AL, 80H

Reads data from port 80H into AL.

9. Example Program

```
ORG 100H
MOV AH, 01H
INT 21H
OUT 80H, AL
MOV DL, AL
MOV AH, 02H
INT 21H
RET
```

Explanation:

1. Takes input from the keyboard
2. Stores it in AL
3. Sends it to port 80H
4. Displays it on screen

Summary

- Assembly language is a low-level language used to control hardware directly
- Registers store temporary data
- Instructions perform operations
- INT 21H is used for input/output
- IN and OUT instructions are used for interfacing